Data Processing and Orchestration

Riccardo Tommasini

Data Engineering 2026 Fall

Slides by Kristo Raun

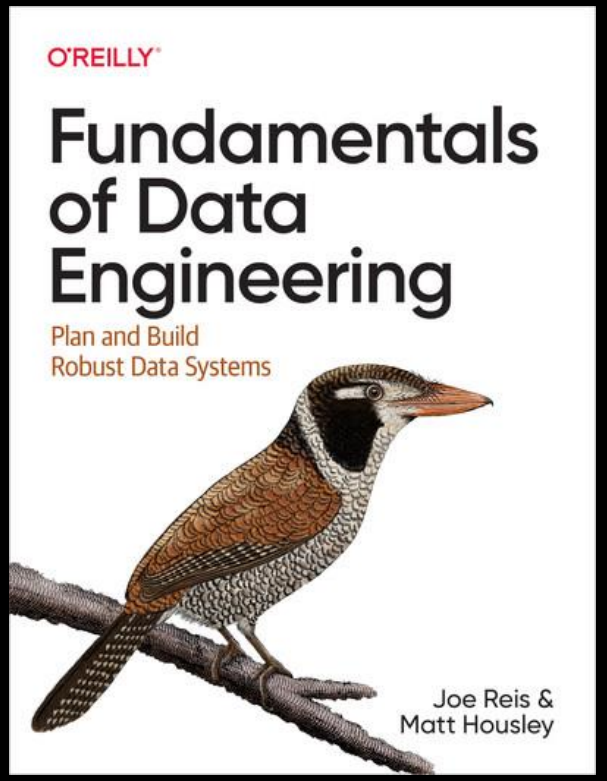
UNIVERSITY OF TARTU

Agenda

- Data processing: ETL & ELT
- Design Patterns
- Data orchestration
- Airflow
- Quiz session

Reading

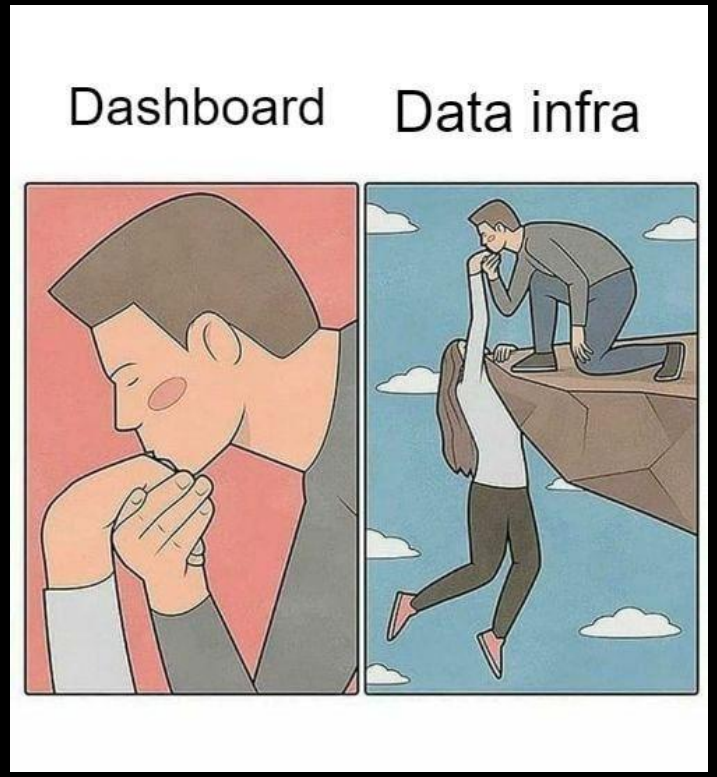
- Chapters V and VII



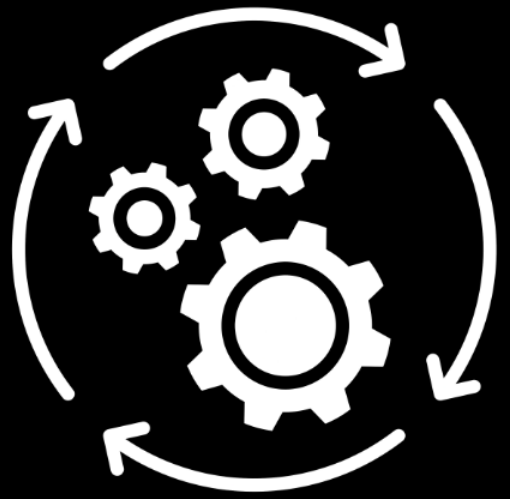
Data processing → what we do with data

Data orchestration → when/how it runs

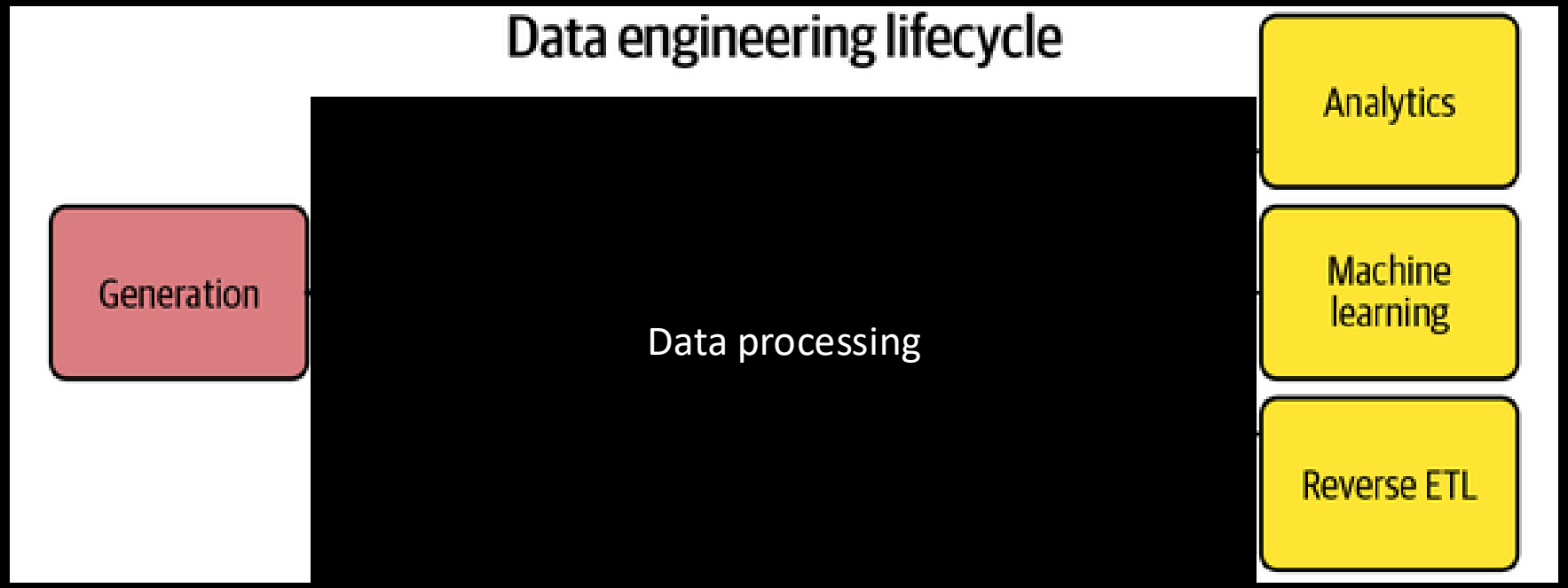
Data automation → runs without you



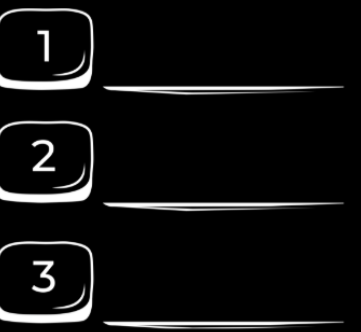
Data processing

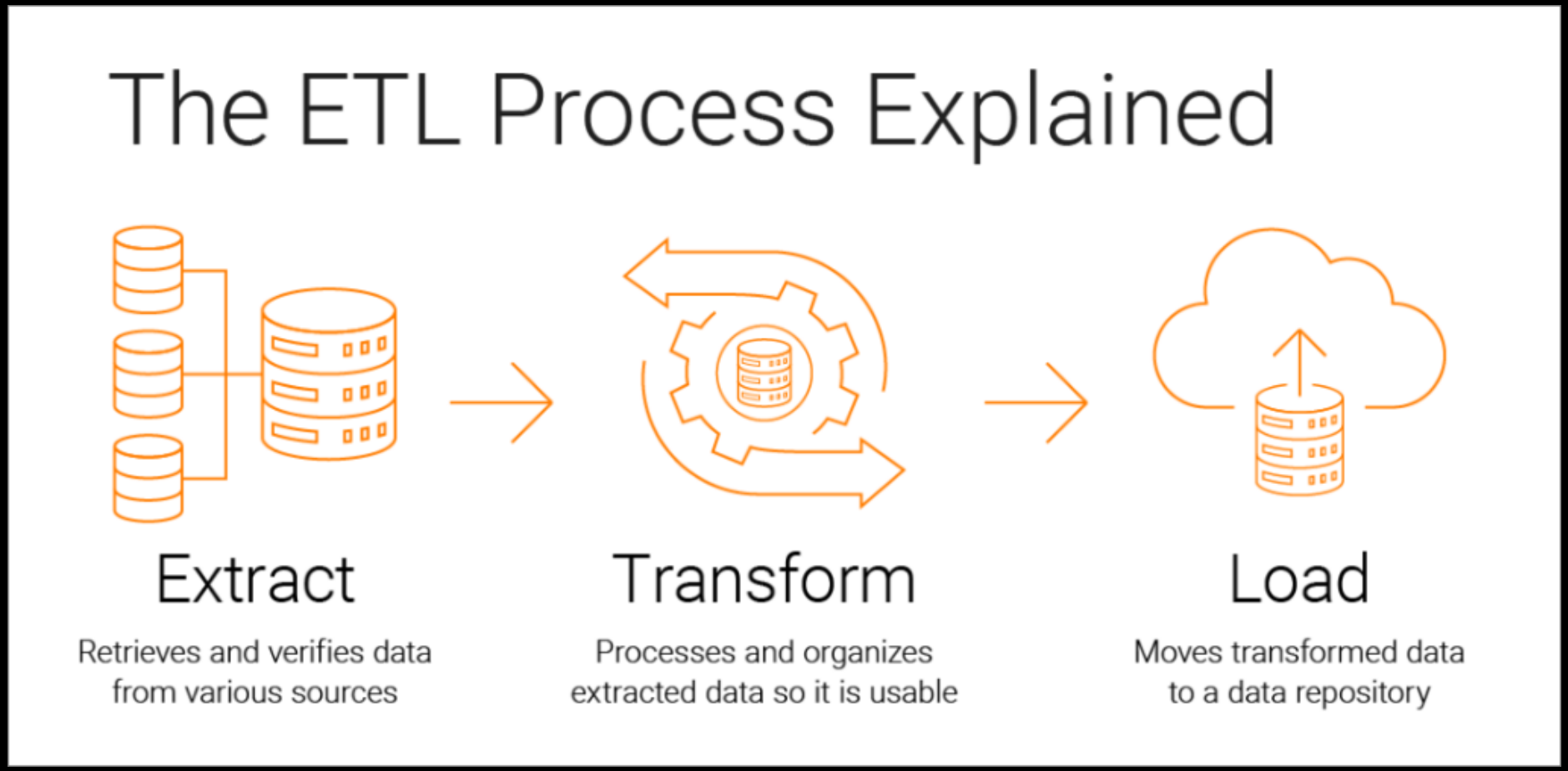


- From raw data
- To usable information



Data processing: ETL

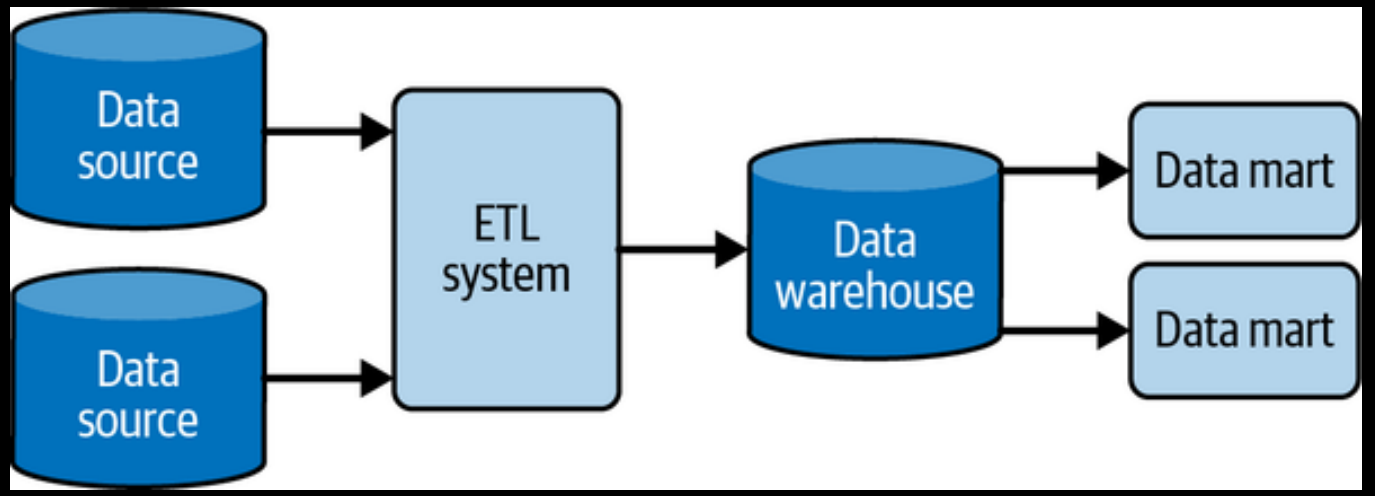




https://www.thedataops.org/list-of-etl-extract-transform-load-

Data processing: ETL

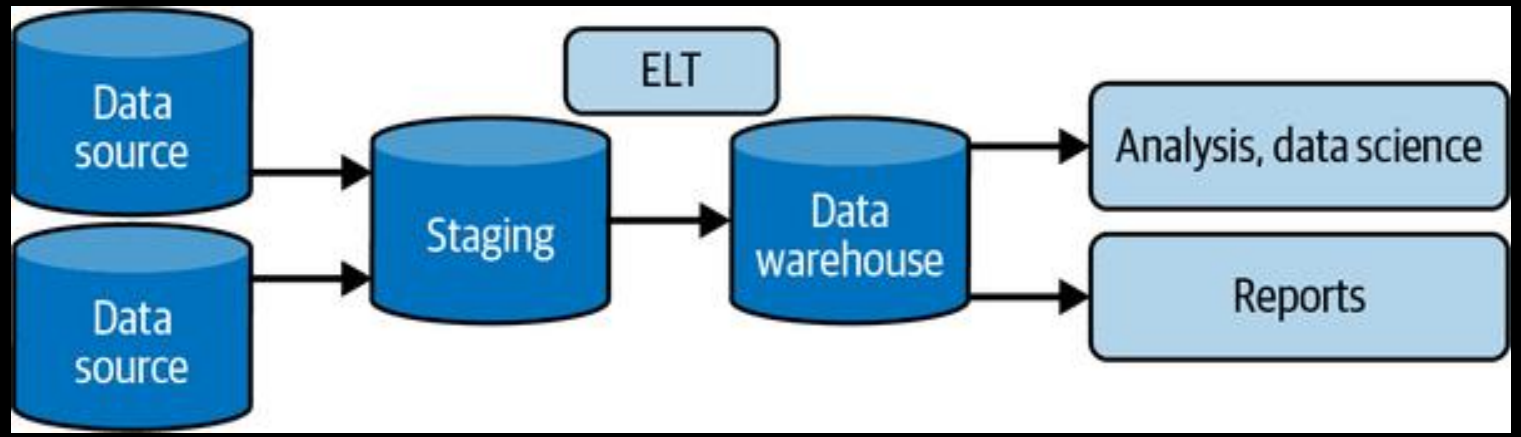
- Classical data warehouses
  - Expensive storage & compute
  - DB engines were slow
  - Network limits



Fundamentals of Data Engineering, ch III

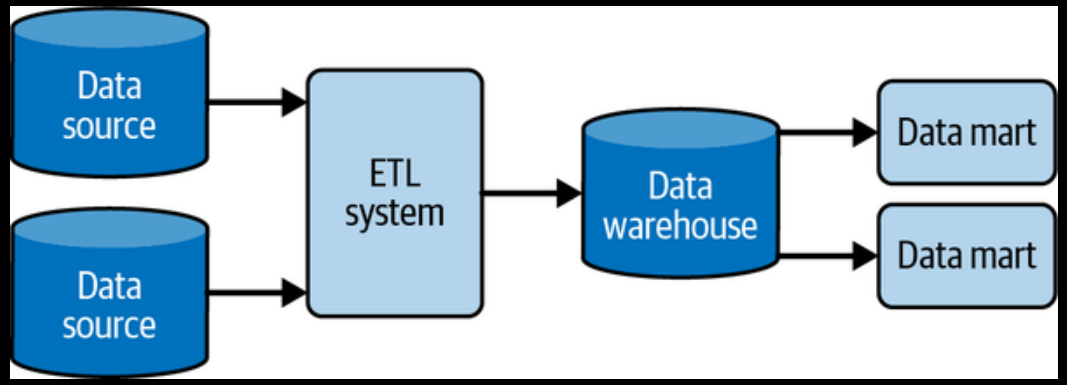
Data processing: ELT

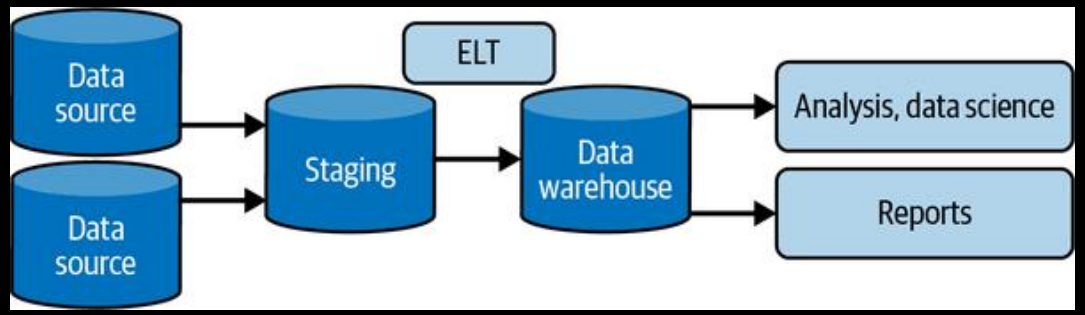
- Cloud data warehouses
  - Cheap and elastic storage
  - Increased processing power
  - Fast network (especially within the same data centre)



Fundamentals of Data Engineering, ch III

Data processing: ETL vs ELT





Fundamentals of Data Engineering, ch III

Data processing: comparison

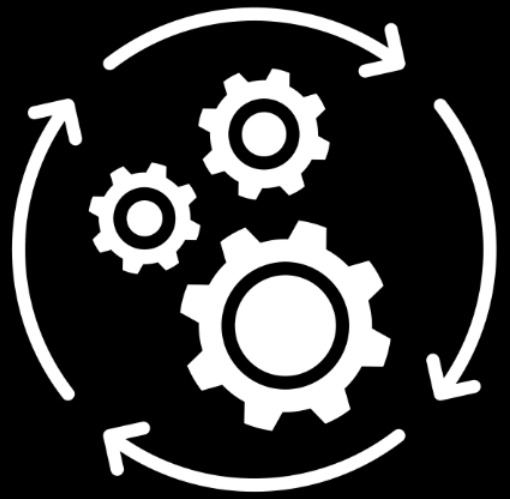


	E-T-L	**E-L-T**
Where does "T" happen?	Outside DWH (ETL system)	Inside DWH (SQL)
Strengths	• Privacy concerns (PII data)	• Highly scalable • Separation of concerns • Fast development • Historical data reuse
Weaknesses	• More moving parts • Slower development time	• Raw data at rest – governance • Compute cost for heavy transformations
Use case	• Regulatory requirements • Raw data unusable (?) • Legacy pipelines	• Any modern cloud platform

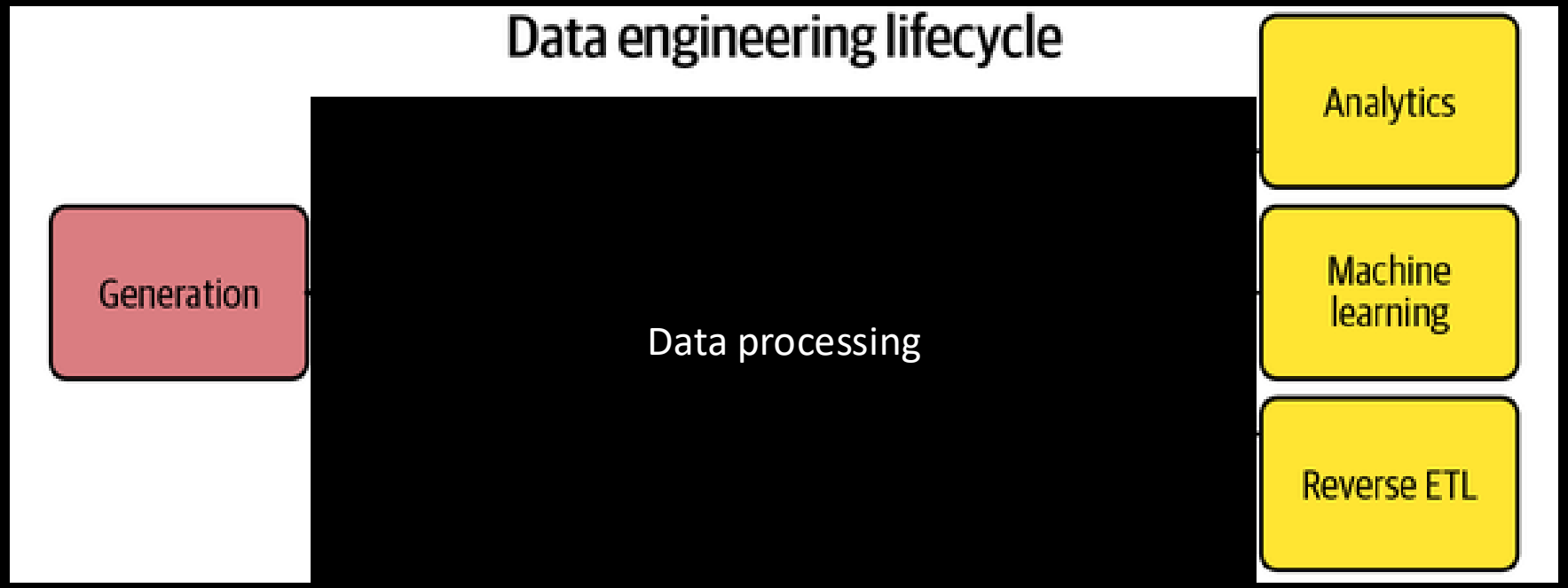
Agenda

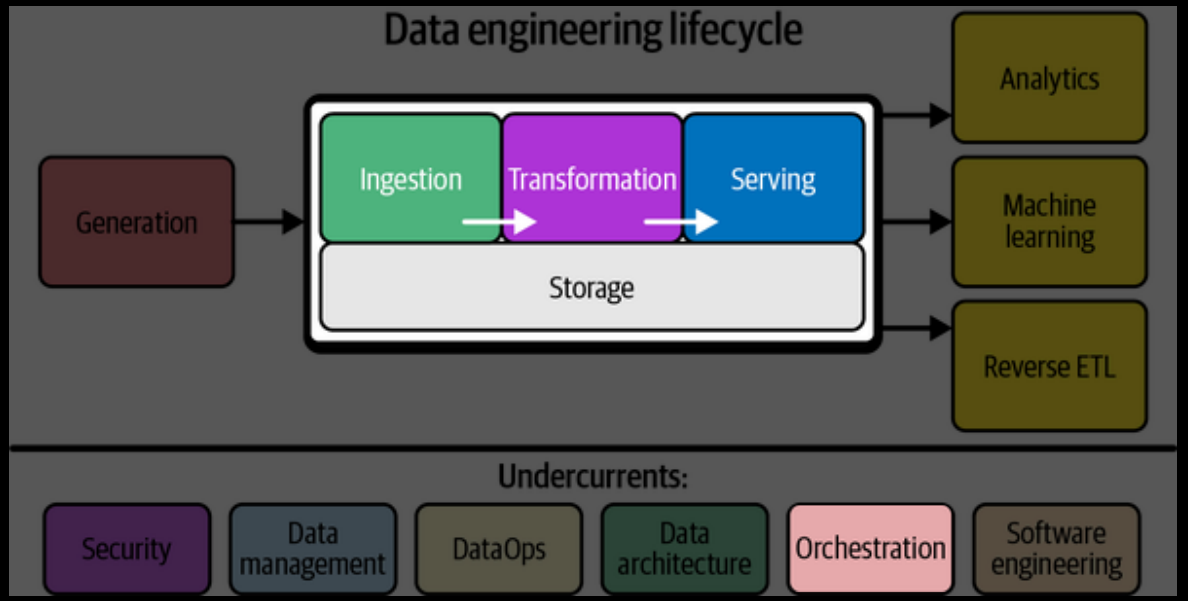
- ~~Data processing: ETL & ELT~~
- Desing Patterns
- Data orchestration
- Airflow
- Quiz session

Data processing



- From raw data
- To usable information




Data engineering lifecycle
Generation
Ingestion
Transformation
Serving
Storage
Analytics
Machine learning
Reverse ETL
Undercurrents:
Security
Data management
DataOps
Data architecture
Orchestration
Software engineering

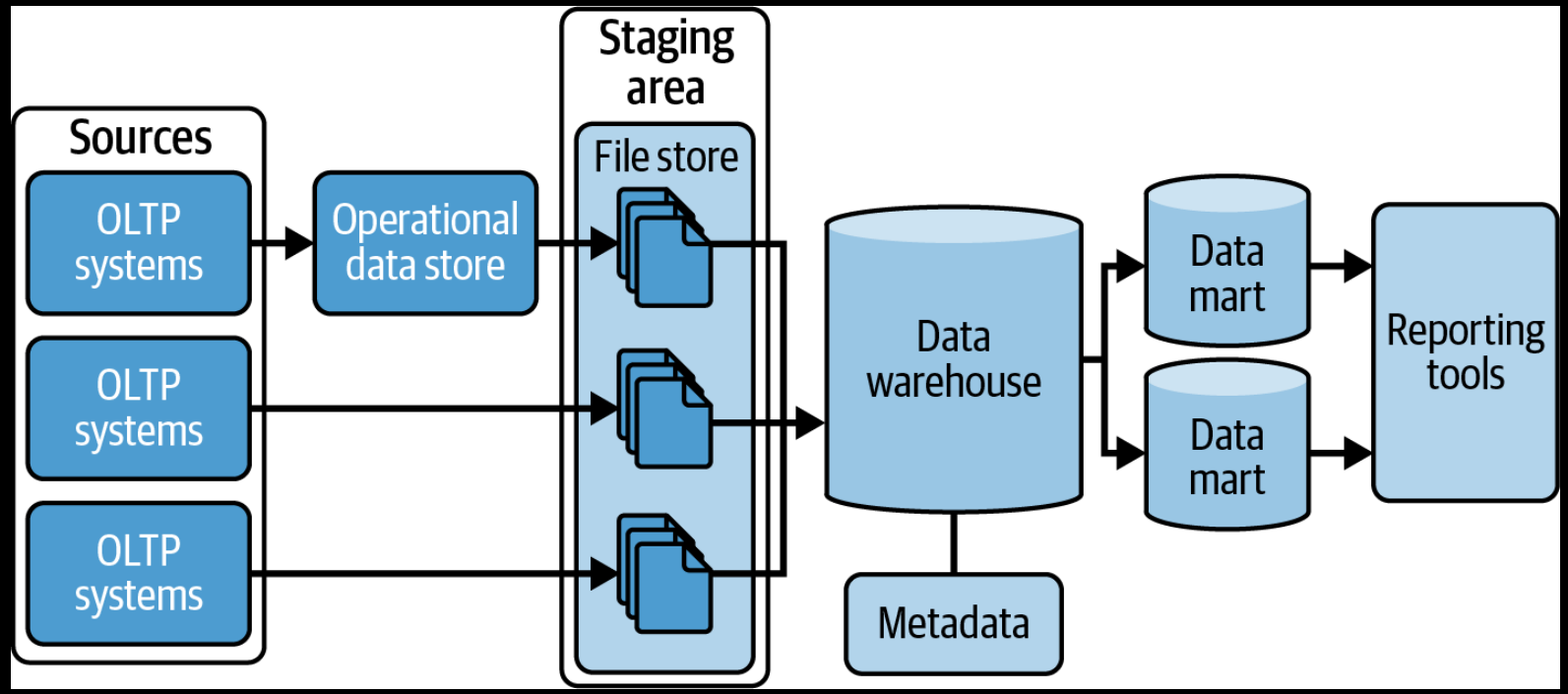
Storage patterns

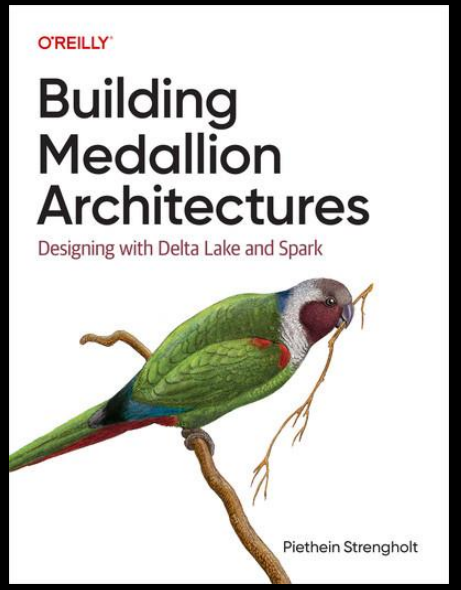


- Database vs data warehouse vs data lake vs data lakehouse
  - Database: typically, OLTP.
  - Others: typically, OLAP
- OLTP vs OLAP
  - OLTP: Online Transaction Processing
  - OLAP: Online Analytical Processing
  - Different query patterns
    - OLTP -> insert, update, delete a single row as fast as possible
    - OLAP -> aggregate a few specific columns across many rows
  - Data “orientation”
    - OLTP: row-based
    - OLAP: column-based
- Schema-on-write vs schema-on-read
  - Database (& DWH) → schema-on-write
  - Data Lake, Data lakehouse → schema-on-read

Storage patterns

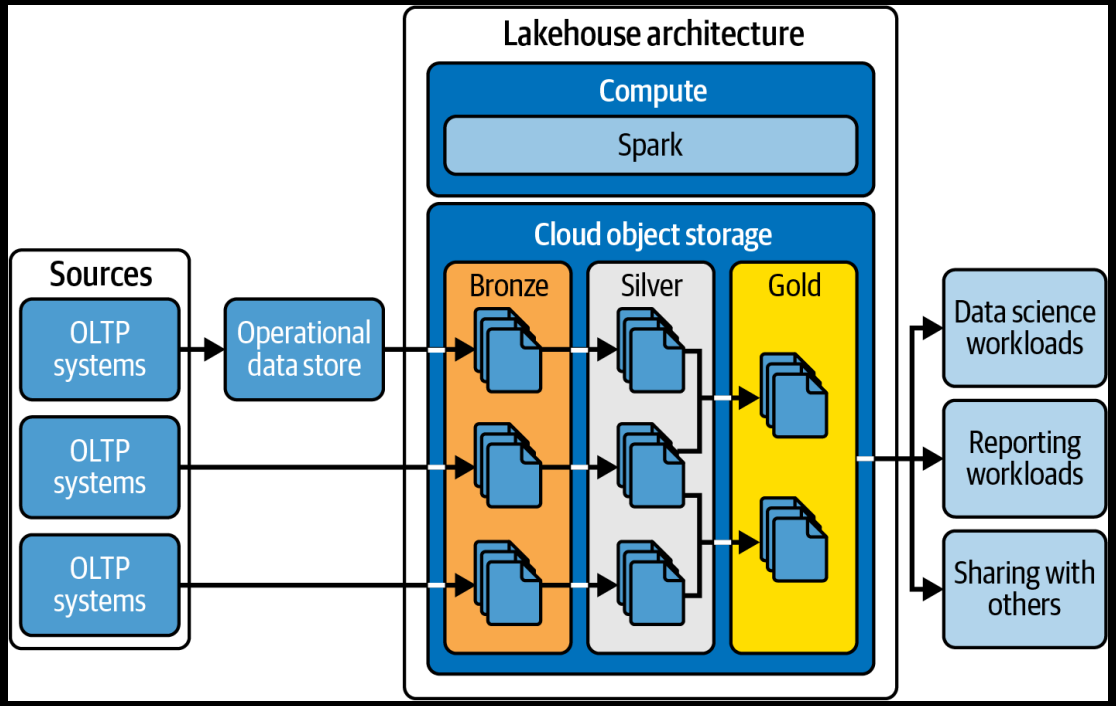
- Layering
  - Typical (traditional) data warehouse architecture:

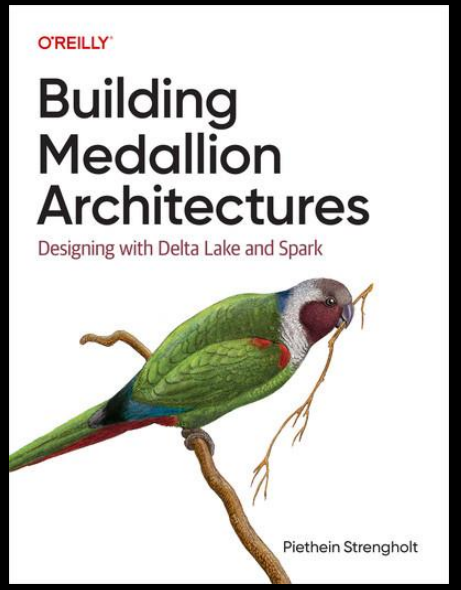




Storage patterns

- Layering
  - Typical lakehouse architecture:





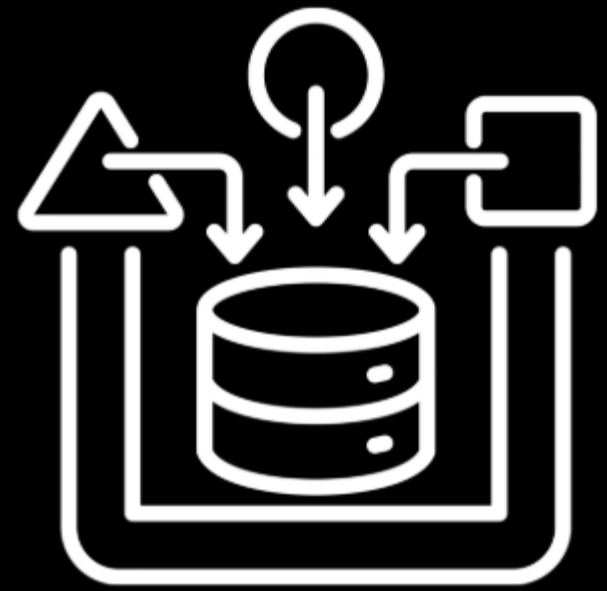
Medallion architecture
Bronze: raw data layer (native structure)
Silver: refined, standardized (improved quality)
Gold: business insights and metrics

Storage patterns



- *Note:* Layering naming conventions...
  - Bronze – silver – gold
  - Raw – conformed – curated
  - Landing – staging – integration – presentation

Ingestion patterns (Extract)



- Batch-based
  - File download
  - SFTP
- API
  - Pull
  - Push (webhook)
- Database-based
  - JDBC connector
  - Change Data Capture (CDC)
- Event-based
  - Object-store events
  - Message queues → Publish & Subscribe (Pub/Sub). Kafka, RabbitMQ

Ingestion patterns (Load)

- Full load + overwrite
- Full load + update (upsert, merge into)
- Incremental load
- Append only

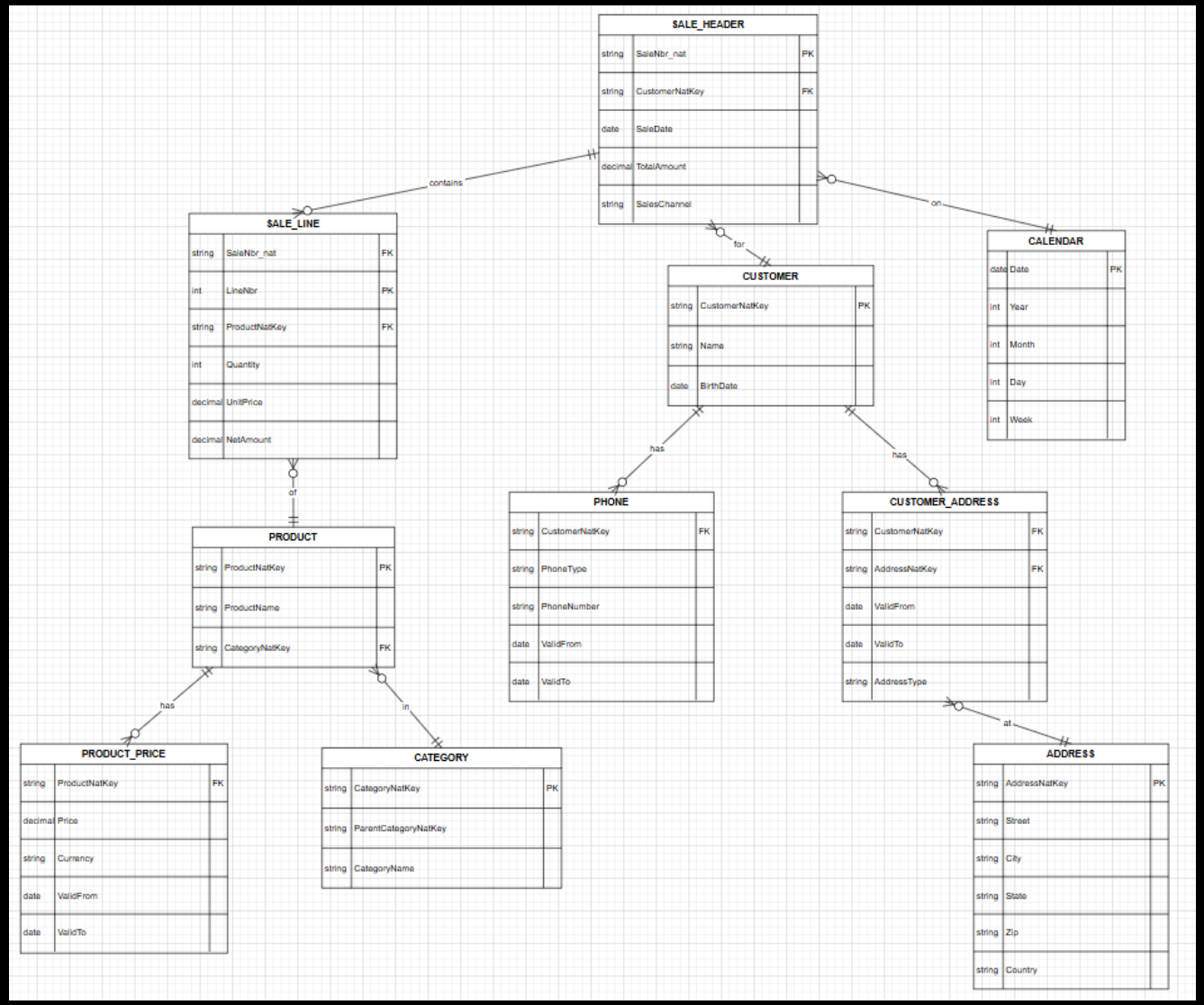


Transformation patterns

- Bill Inmon
  - Top-down approach
  - Corporate Information Factory (CIF)
  - Enterprise Data Warehouse (EDW)
- Data Vault (Dan Linstedt)
  - Hub-and-Spoke
  - EDW 2.0
- Ralph Kimball
  - Bottom-up approach
  - Dimensional modeling
  - Star schema (snowflake schema)
  - Facts and dimensions
- One Big Table (OBT)
  - Wide tables

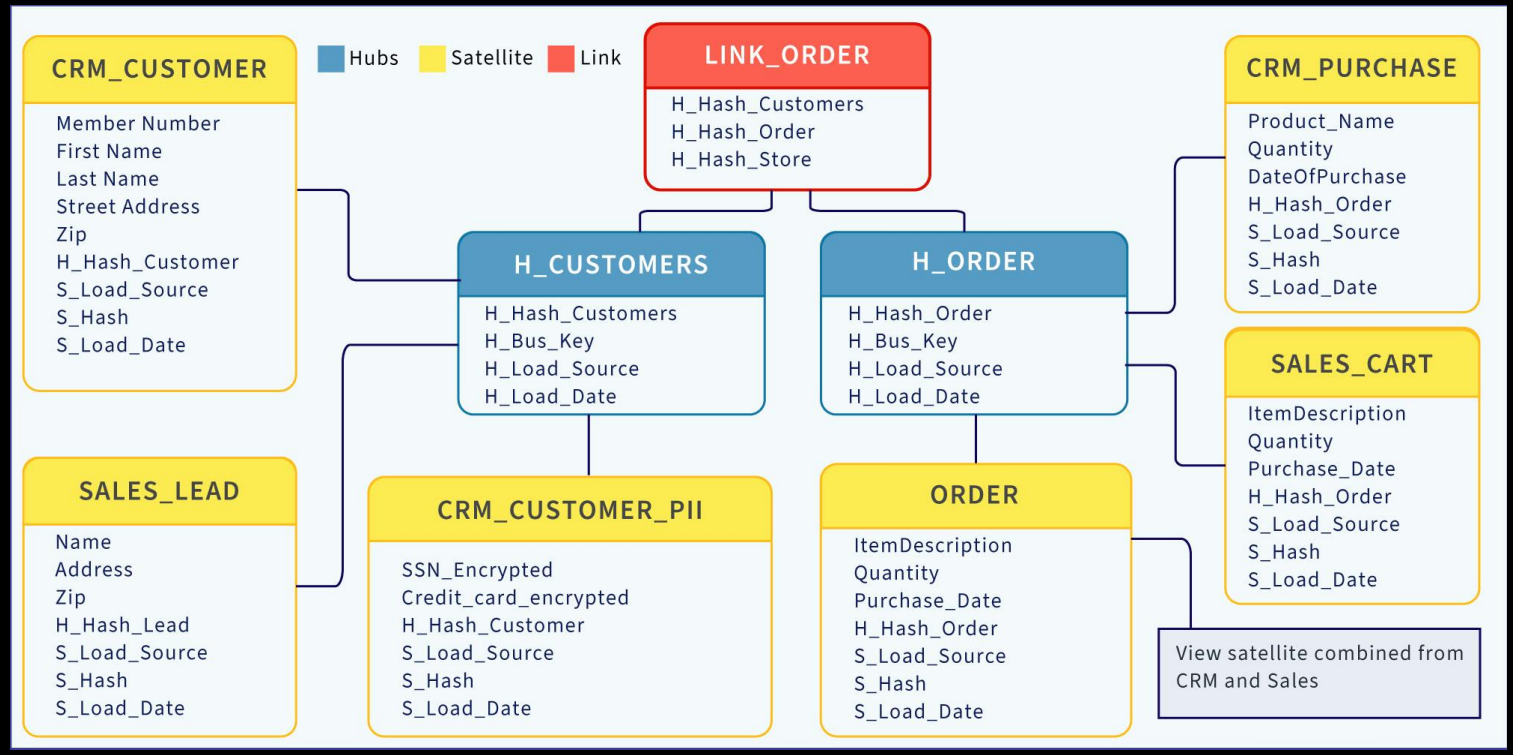
Transformation patterns

- Inmon
  - 3NF
  - Top-down approach
  - Good scalability
- Note:
  - Not so much used anymore



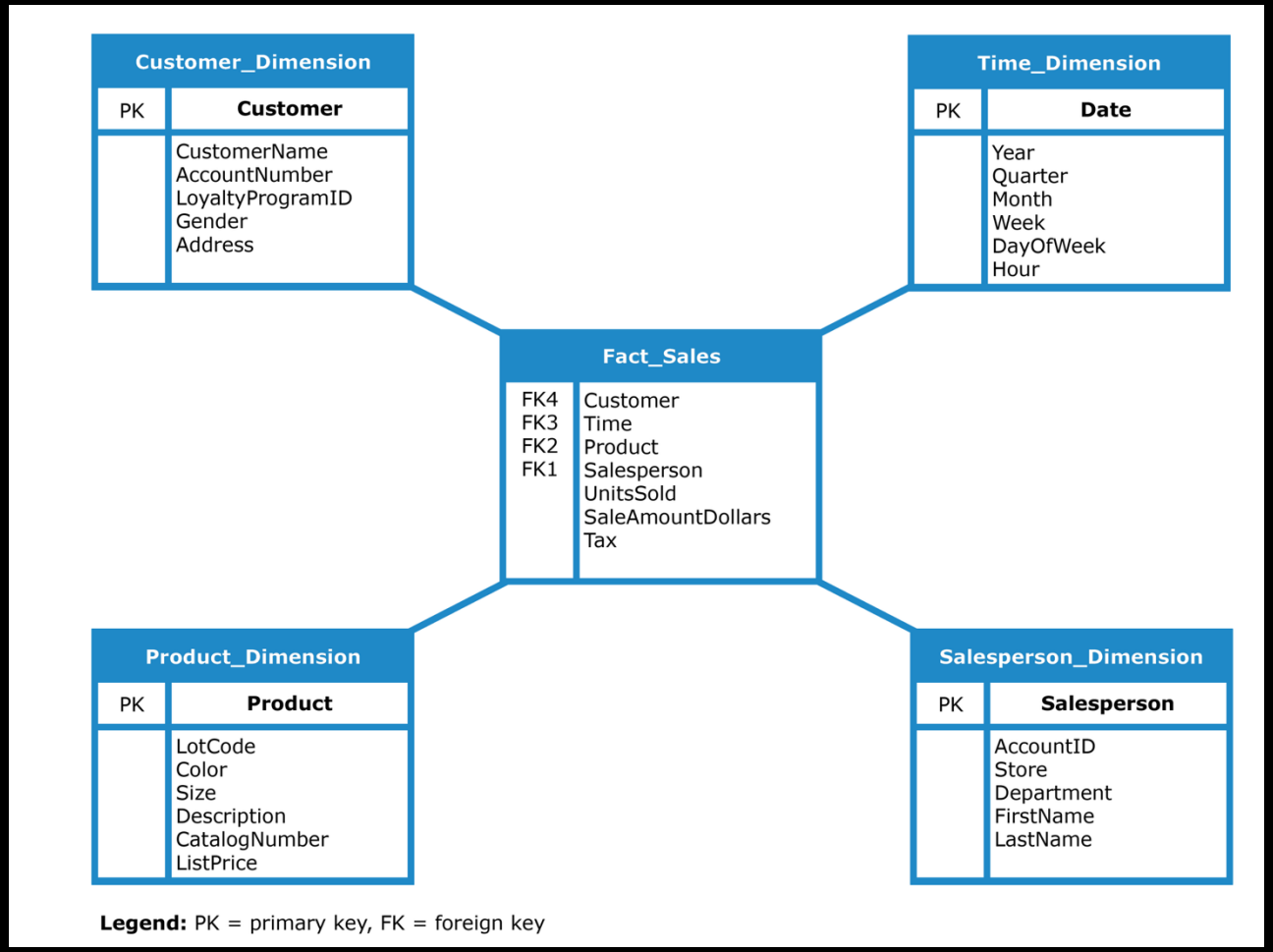
Transformation patterns

- Data Vault
  - Insert only
  - Auditable
  - Adaptable
- Hub
  - What entity?
- Link
  - How do entities interact?
- Satellite
  - Descriptions of entities and interactions



Transformation patterns

- Kimball
  - Denormalized
  - Bottom-up approach
  - Focus on delivering value



Transformation patterns

- OBT (One Big Table)
  - Delivering value fast: on steroids
  - Fast to setup
  - Fast to query (on modern, columnar data stores)

SALES_OBT	
int	SaleID
string	CustomerName
string	Address
string	PhoneNumber
string	ProductName
string	Category
decimal	Price
date	SaleDate
int	Year
int	Month
int	Day
string	Quarter
decimal	SaleAmount
int	Quantity
string	CustomerSegment
string	Supplier
string	AdditionalInfo

Transformation patterns

Feature	**Inmon (3NF)**	**Data Vault**	**Kimball (Star Schema)**	**One Big Table (OBT)**
Approach	Top-Down	Hybrid (Hub-and-Spoke)	Bottom-Up	Flat, Denormalized
Structure	Multiple Normalized Tables (3NF)	Hubs, Links, Satellites	Fact Table + Dimension Tables (Star Schema)	Single Wide Table
Data Redundancy	Minimal Redundancy	Minimal to Moderate Redundancy	Allows Redundancy for Simplicity	High Redundancy
Query Complexity	High (Multiple Joins)	Moderate (Joins between Hubs, Links, Satellites)	Low (Fewer Joins)	Very Low (Single Table Queries)
Flexibility	Highly Flexible and Scalable	Highly Flexible for Historical Tracking	Optimized for Specific Query Performance	Less Flexible for Schema Changes
Use Case Focus	Enterprise-Wide Integration	Scalable, Historical Data Tracking	Departmental Reporting and Analysis	High-Performance Reporting and Analytics

Serving patterns

- Ways of distributing data → See ingestion patterns



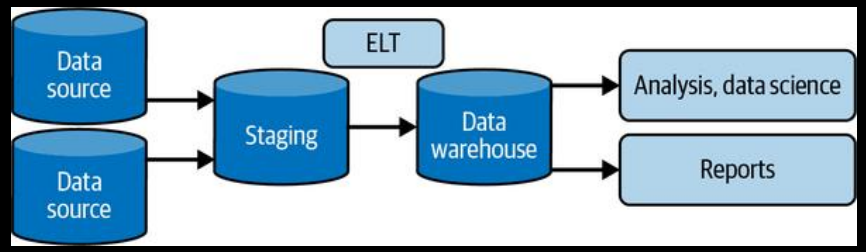
- Data marts
  - Standard BI reporting
- OBT
  - Simple view for business/data science
- Semantic layer
  - Matched to business glossary
- Feature store
  - Features for ML models

Agenda

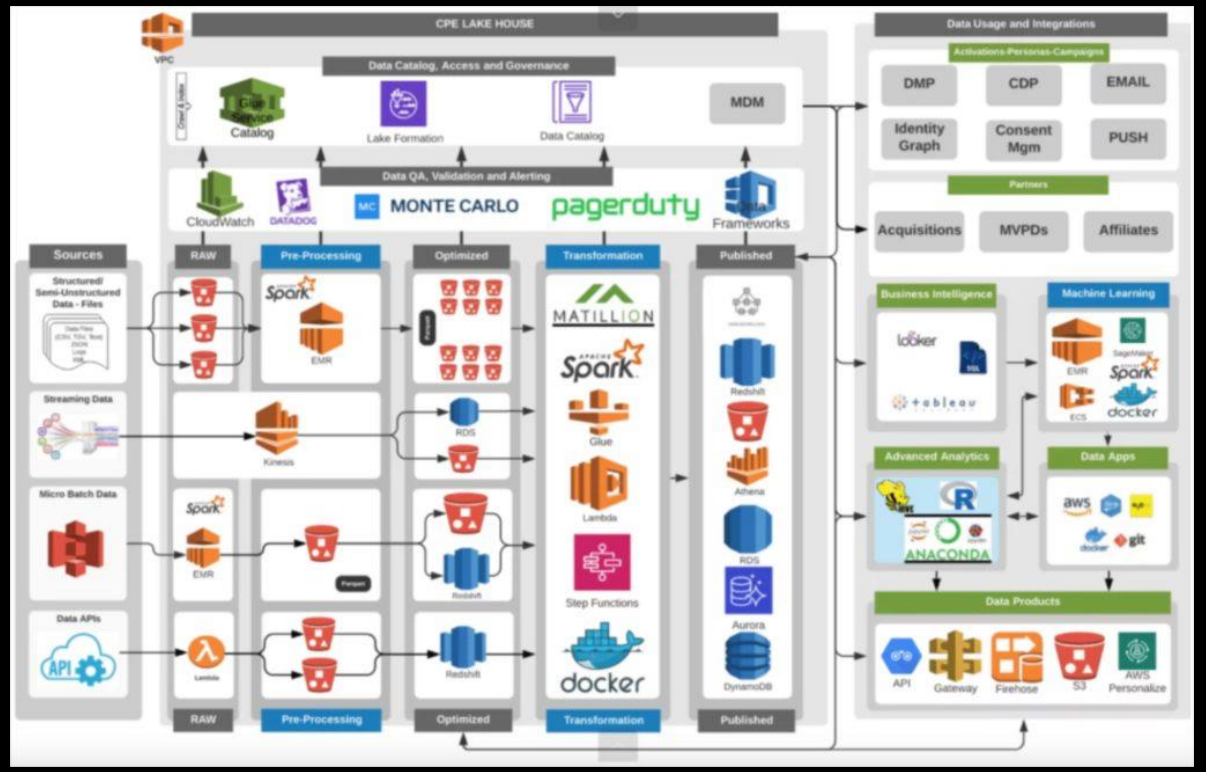
- ~~Data processing: ETL & ELT~~
- ~~Desing Patterns~~
- Data orchestration
- Airflow
- Quiz session

Why data orchestration?

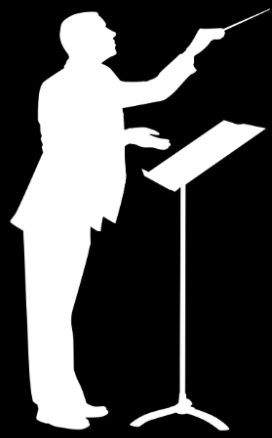
- Real-world data pipelines are complex

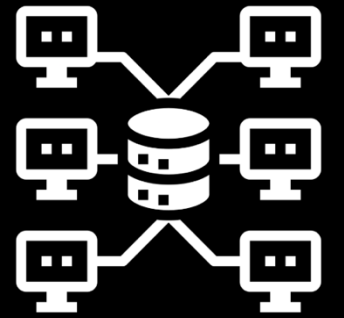


VS



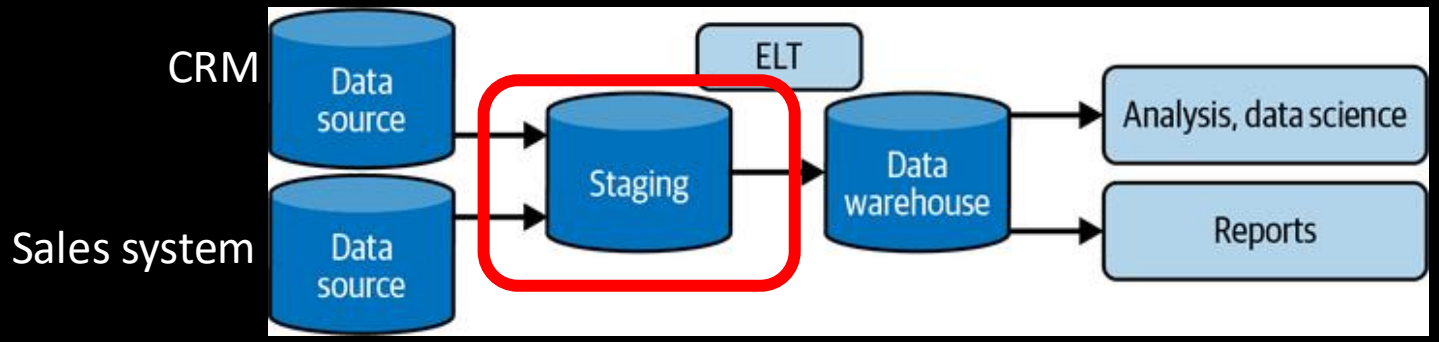
Data orchestration





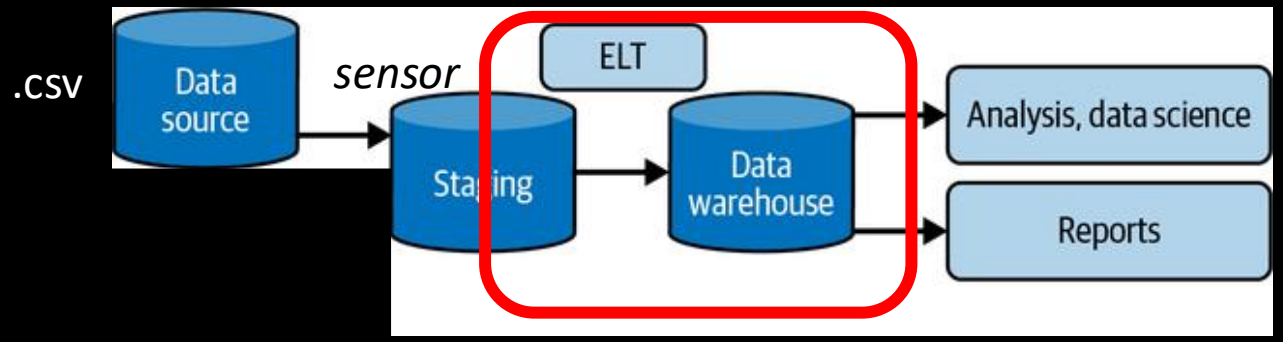
- Coordinating many jobs
  - Real-world pipelines are not isolated
- Considerations
  - Idempotency & backfills
    - Safe re-runs over a historical period
  - Retries, timeouts, and alerts
    - Handling API errors, usage spikes, etc
  - Concurrency/resource limits
    - Both in source system and your target system
  - Data quality gates
    - Fail fast for breaking changes
  - Parameterization & calendars
    - Execute only for date X system Y
  - Secrets & access
    - *Critical!* Use service accounts, key rotation, do NOT commit secrets to git
  - Lineage & observability
    - Enable (verbose) logs, metrics, owners & SLAs (who to alert)
  - Task vs data dependencies
    - Task dependency: task B follows task A (smaller teams, coupled systems)
    - Data dependency: task B triggered when data in table T (distributed teams, decoupled systems)

Data orchestration – quick use case 1



- Sales report
  - Sources:
    - CRM
    - Sales system
  - Both need to be loaded to DWH (staging) before transformations can be applied

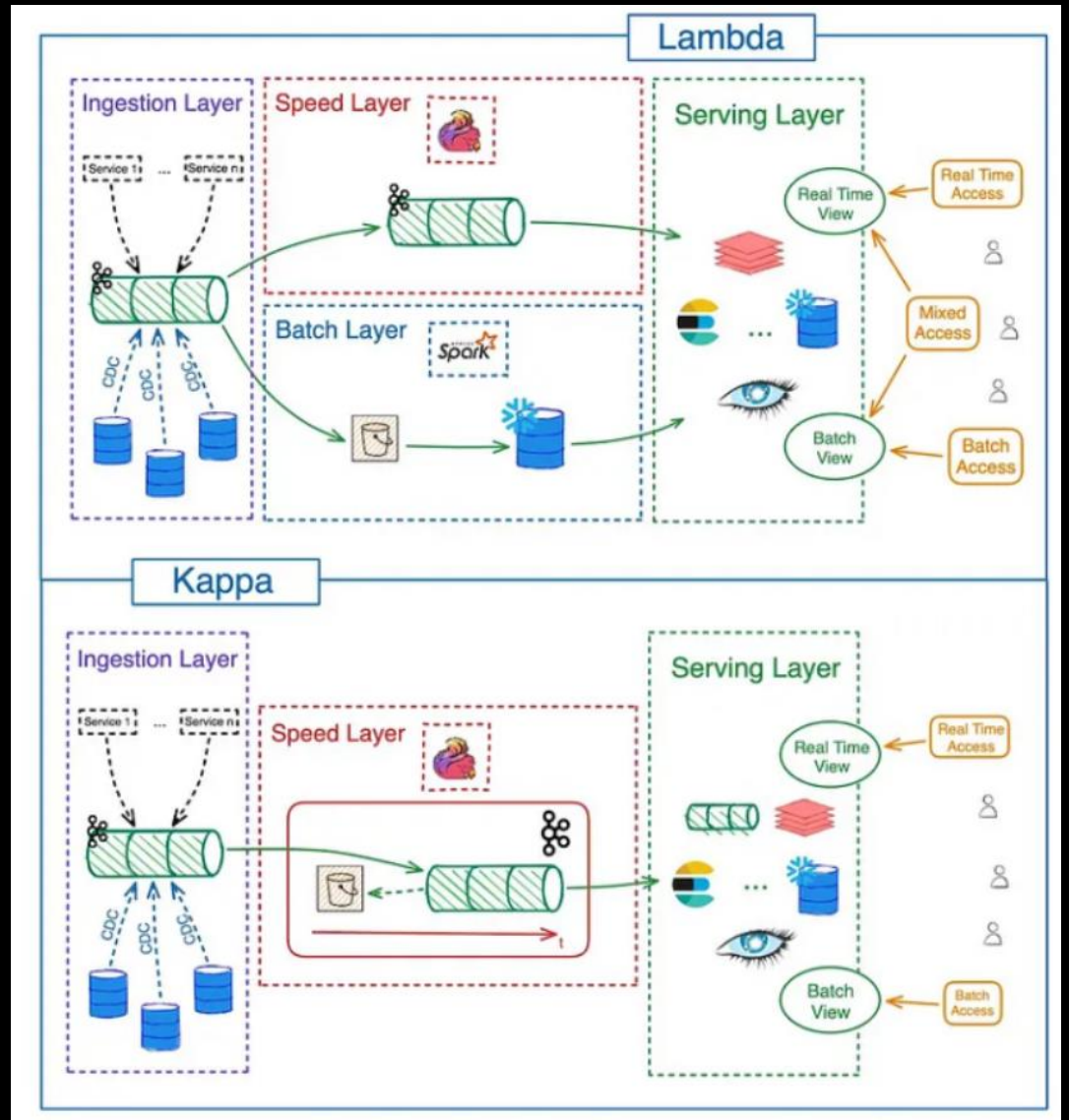
Data orchestration – quick use case 2



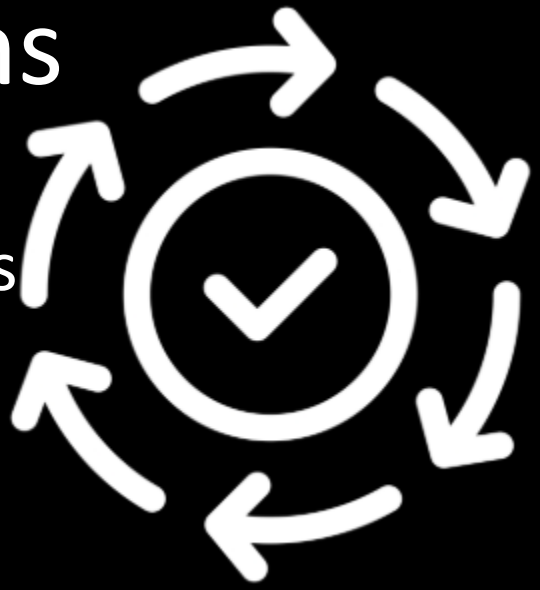
- External report upload
  - Source:
    - .csv uploaded to object storage daily
    - The file upload is controlled by the external vendor
  - The rest of the workflow should only start when file has been uploaded

Processing & Orchestration Patterns

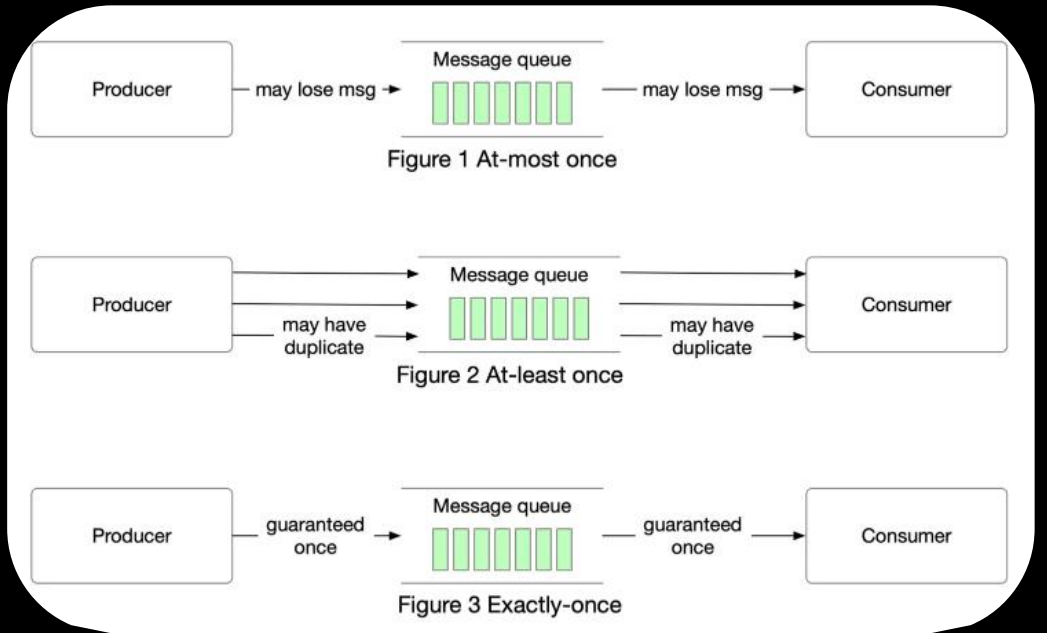
- Batch vs Streaming
  - Batch: every X interval
  - Streaming: continuous, near real-time
  - (Micro-batch: few second batch interval)
- Lambda vs Kappa



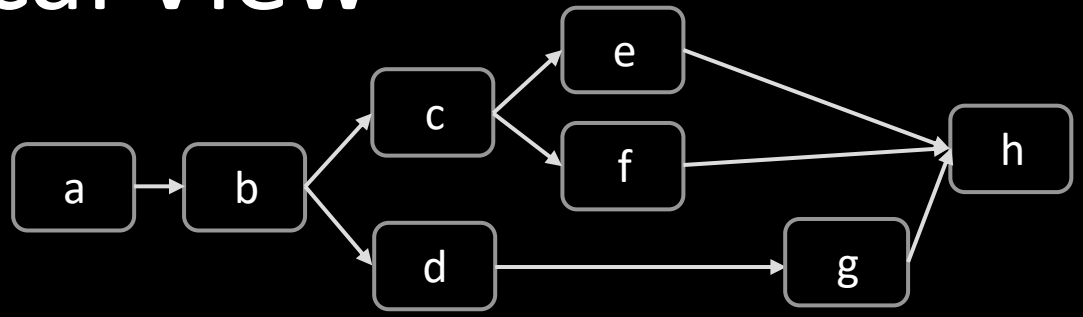
Processing & Orchestration Patterns



- Orchestration: large pipelines, complex dependencies
  - Time-based. Simplest: CRON
  - Event-driven (“sensor”)
  - Dependency-based (DAG – Directed Acyclic Graph)
  - Manual triggers (backfills)
- Streaming perspective
  - At most once
  - At least once
  - Exactly once
  - *Note: idempotency!*



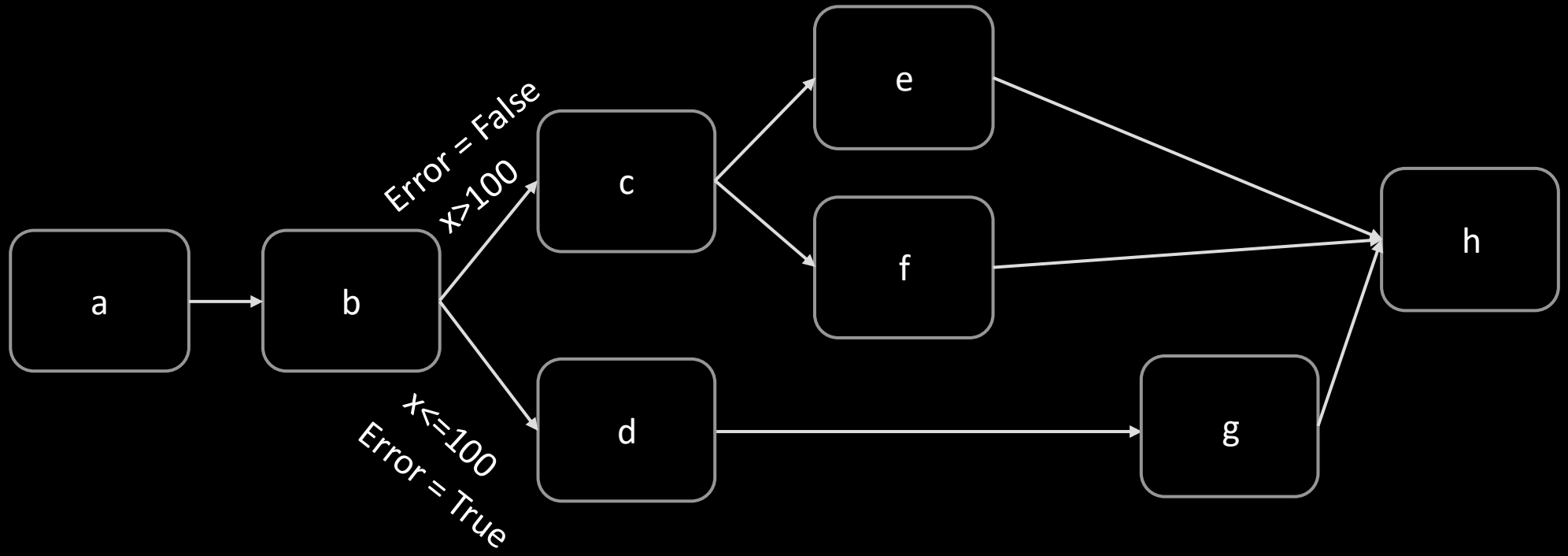
Data orchestration – Logical View



- DAG
  - Directed
    - Determines task orders and dependencies
  - Acyclic
    - You can't loop back to an already completed task (avoids paradoxes, infinite loops)
    - Loops happen across runs (not within a single run)
- Control flow:
  - Sequential
  - Parallel
  - Conditional (branching)
  - *Various subtypes depending on the tool*

Note on DAG visuals

- Parallel (AND) vs branching (XOR) in DAG?



Agenda

- ~~Data processing: ETL & ELT~~
- ~~Desing Patterns~~
- ~~Data orchestration~~
- Airflow
- Quiz session

Airflow



- Airflow is a platform created by the community to programmatically author, schedule and monitor workflows.
- Origins: 2014 @Airbnb
- Data engineers can write orchestration in **a single Python file**

https://airflow.apache.org/

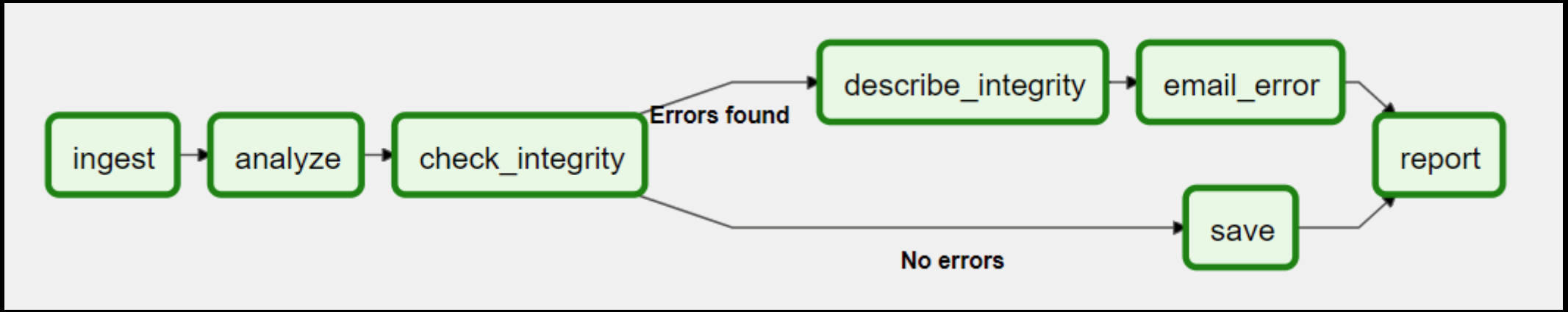
Why Airflow?

- “ We need a new [*dashboard, external API, data warehouse, ML pipeline, …]”
- How will we make the job run automatically once per day?
- Task A, takes ~ 1h
- Task B, takes ~ 30 min
- Task C, takes ~ 30 min, should run only if Task A has finished successfully
- Task D, takes ~ 20 min, should run after file “prevDay.csv” is uploaded to “/tmp/taskD”

- Can we still use our current scheduling tool?

About

- Running a workflow in Airflow is represented as a **DAG**

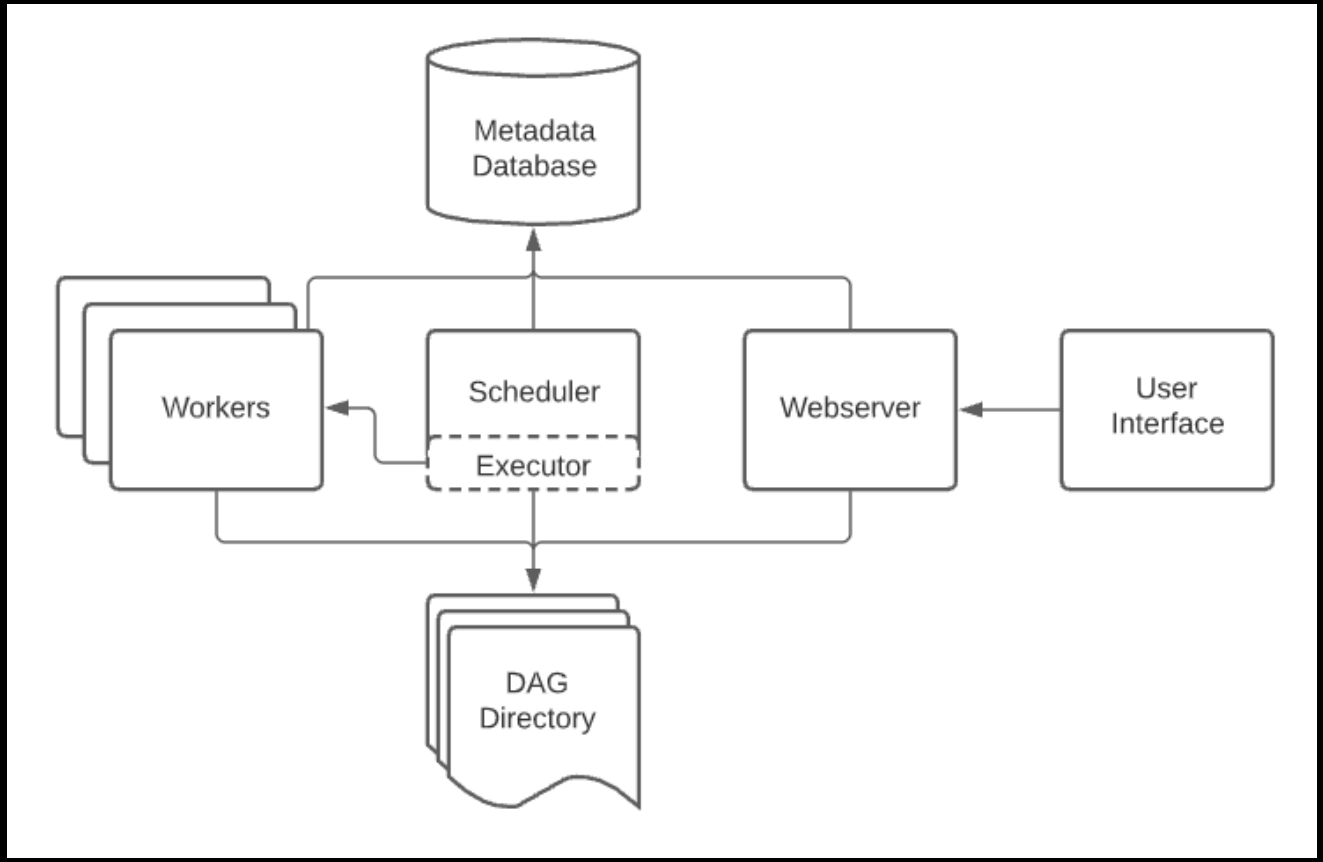


- A DAG specifies the dependencies between Tasks, and the order in which to execute them and run retries;
- The Tasks themselves describe what to do, be it fetching data, running analysis, triggering other systems, or more.

Source: https://airflow.apache.org/docs/apache-airflow/stable/concepts/overview.html

Architecture

- Scheduler
  - triggering scheduled workflows
  - submitting Tasks to the executor to run
- Executor
  - handling the running of tasks
- Webserver
  - UI to inspect, trigger and debug the behaviour of DAGs and tasks
- A folder of DAG files
  - read by the scheduler and executor (and any workers the executor has)
- A metadata database
  - used by the scheduler, executor and webserver to store state.



Airflow dictionary

- DAG
  - “workflow”
- Task
  - A basic unit of execution
- Operator
  - A template for a task. *PostgresOperator, PythonOperator, HiveOperator, …*
- Sensor
  - An operator for waiting for something. *FileSensor, SqlSensor, ExternalTaskSensor*

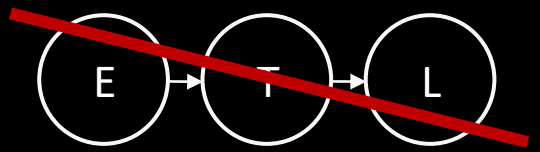
Airflow: control plane vs data plane

- Control plane: orchestrator
  - Handle code, schedule, permissions, secrets
  - Trigger executors
- Data plane: processes data
  - Where compute happens (database, Python script, S3, Kafka, ...)
  - Handles actual data movement and manipulation
- Airflow → mainly control plane, triggering data plane operators

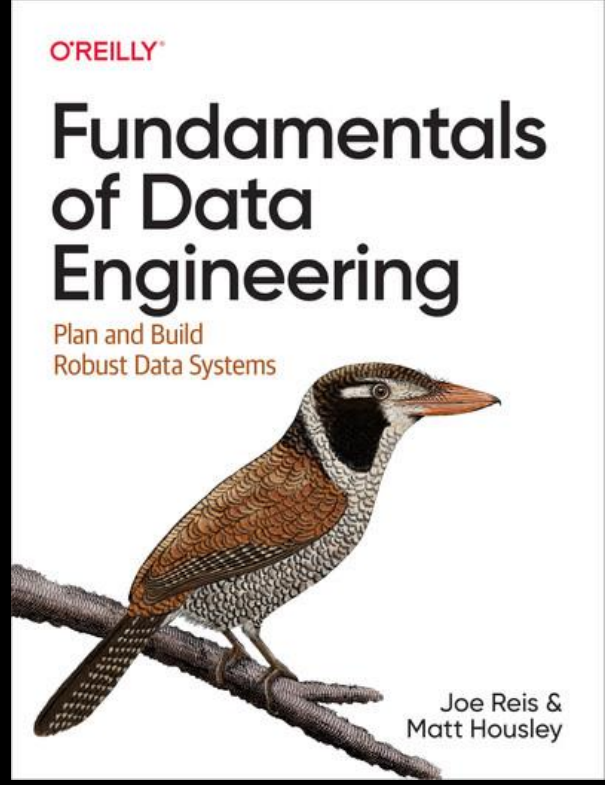
Final words…

- Airflow is great for organizing complex workflows
- Airflow is not an *ETL* tool





Read more:



- Chapter V: Data generation in source systems
  - How is data created?
  - Types of data in source systems
- Chapter VII: Ingestion
  - Batch ingestion considerations
  - Ways to ingest data
  - “At times, the minutiae of ingestion may feel tedious, but the exciting data applications (e.g., analytics and ML) cannot happen without it.”
- Why Data Engineers LOVE/HATE Airflow (by Seattle Data Guy)
  https://www.youtube.com/watch?v=h5X3124R61U